학습 교안

하네스 엔지니어링 입문 2단계

AI 에이전트와 함께 규칙, 기준, 변경, 되돌리기를 익힌다 도서 대출 목록 프로그램 하나

OpenCode · OpenRouter · Python · pytest · Git

준비하기 + 1~7장 · 1단계 이후의 초급 실습

강사 김대철

작성일 2026.11.13

실습 환경 Windows + Python + Git + OpenCode

실습 저장소 https://github.com/sinaboro/harness-beginner-tutorial

교안의 명령과 테스트 결과는 Windows PowerShell 에서 실제로 실행해 얻은 것입니다.
AI 의 답변은 실행할 때마다 달라지므로, 교안에는 답변 대신 확인 방법을 실었습니다.

목차

과정 안내 . . . 5

단계 안내 . . . 5
2단계 교시표 . . . 5
도커로 실행하기 (선택) . . . 5
0단계 교시표 . . . 6
1단계 교시표 . . . 6
저장소 구조 . . . 7
시작하기 . . . 7
원칙 세 가지 . . . 7
강사 안내 . . . 8

2단계 준비하기 (도커) . . . 9

1. 컨테이너 켜기 (Windows) . . . 9
2. 도구 확인 (컨테이너 안) . . . 9
3. 작업 폴더 만들기 (컨테이너 안) . . . 9
4. 1단계 도구 복사하기 (컨테이너 안) . . . 9
확인 체크리스트 . . . 10

2단계 1장. 도서관 요구사항 쓰기 . . . 11

학습 목표 . . . 11
1. 개념 — 규칙은 상태에서 나온다 . . . 11
2. 따라 하기 . . . 11
3. 확인하기 . . . 12
4. 자주 만나는 오류 . . . 12
5. 확인 문제 . . . 12

2단계 2장. 책 목록을 파일에 저장하기 . . . 13

이 장에서 할 일 한눈에 보기 . . . 13
1단계. 테스트 기준 파일 가져와 읽기 (15분) . . . 13
2단계. 빨강 확인하기 (10분) . . . 13
3단계. AI 에게 지시서 건네고 계획 검토하기 (25분) . . . 14
4단계. 초록 확인하고 테스트 파일 대조하기 (15분) . . . 14
5단계. 저장된 파일 눈으로 확인하기 (15분) . . . 15
6단계. 저장하기 (10분) . . . 15
확인 체크리스트 . . . 15
자주 만나는 오류 . . . 16

2단계 3장. 대출과 반납: 작은 지시로 나눠 맡기기 . . . 17

이 장에서 할 일 한눈에 보기 . . . 17

1단계. 상태 규칙 테스트 가져와 읽기 (15분) . . . 17
2단계. 빨강 확인하기 (10분) . . . 18
3단계. AI 에게 지시서 건네고 계획 검토하기 (25분) . . . 18
4단계. 초록 확인하고 테스트 파일 대조하기 (15분) . . . 18
5단계. 상태 규칙을 직접 실행해서 확인하기 (20분) . . . 19
6단계. 저장하기 (10분) . . . 19
확인 체크리스트 . . . 19
자주 만나는 오류 . . . 20

2단계 4장. AI 가 만든 실수 찾기 . . . 21

이 장에서 할 일 한눈에 보기 . . . 21
1단계. 내 완성본 보관하고 초안으로 바꾸기 (10분) . . . 21
2단계. 테스트로 실수 찾기 (15분) . . . 21
3단계. AI 에게 고치게 하기 (20분) . . . 22
4단계. 테스트가 못 잡은 실수 직접 찾기 (20분) . . . 22
5단계. 빠진 테스트를 사람이 쓰고, AI 에게 고치게 하기 (25분) . . . 23
6단계. 내 완성본과 비교하고 저장하기 (15분) . . . 23
확인 체크리스트 . . . 24
자주 만나는 오류 . . . 24

2단계 5장. 요구사항이 바뀌었을 때 . . . 25

이 장에서 할 일 한눈에 보기 . . . 25
1단계. 변경 전 상태 확인하기 (10분) . . . 25
2단계. 변경 요청서 칸별로 채우기 (25분) . . . 25
3단계. 영향을 받는 테스트 찾기 (10분) . . . 26
4단계. 승인하고 테스트 기준을 사람이 바꾸기 (20분) . . . 26
5단계. 바뀐 기준으로 빨강 확인하기 (10분) . . . 27
6단계. AI 에게 구현시키고 대조하기 (30분) . . . 27
7단계. 저장하기 (15분) . . . 28
확인 체크리스트 . . . 28
자주 만나는 오류 . . . 28

2단계 6장. 잘못된 변경을 되돌리기 . . . 29

이 장에서 할 일 한눈에 보기 . . . 29
1단계. 요구사항에 없는 기능 시키기 (20분) . . . 29
2단계. 범위를 대조해서 잘못 찾기 (15분) . . . 29
3단계. 저장 전 변경을 git restore 로 되돌리기 (15분) . . . 30
4단계. 저장 후 변경을 git revert 로 되돌리기 (30분) . . . 30
5단계. 되돌린 뒤 검증하고 기록 확인하기 (20분) . . . 31
6단계. 되돌리기 방법 고르기 (표 채우기, 15분) . . . 31
7단계. 정리하고 확인하기 (10분) . . . 32

확인 체크리스트 . . . 32
자주 만나는 오류 . . . 32
2단계 7장. 미니 프로젝트: 도서 대출 목록 완성 . . . 33
이 장에서 할 일 한눈에 보기 . . . 33
1단계. 새 작업 폴더 만들고 기준 파일 가져오기 (15분) . . . 33
2단계. 요구사항과 테스트 이름 대조하기 (20분) . . . 34
3단계. AI 에게 맡기고 계획 검토하기 (25분) . . . 34
4단계. 검증하고 테스트 파일 대조하기 (15분) . . . 35
5단계. 직접 실행해서 확인하기 (20분) . . . 35
6단계. 기록 쓰고 저장하기 (25분) . . . 36
확인 체크리스트 . . . 37
막혔을 때 . . . 37
다음 단계 . . . 37
부록. 템플릿과 예제 . . . 38
A-1. 도서 대출 요구사항 최종본 (7장) . . . 38
A-2. 요구사항 변경 요청서 (5장) . . . 39
A-3. 책 저장 작업 지시서 (2장) . . . 39
A-4. 대출과 반납 작업 지시서 (3장) . . . 40
A-5. 미니 프로젝트 작업 지시서 (7장) . . . 40
A-6. 책 등록과 저장 테스트 (2장) . . . 41
A-7. 대출과 반납 테스트 1차 (3장) . . . 42
A-8. 대출과 반납 테스트 2차 (5장) . . . 43
A-9. 4장 연습용 초안 (검증 전) . . . 45

과정 안내

AI 에이전트에게 개발을 맡기되, 규칙·권한·검증으로 통제하고, 통과한 것만 저장하는 법을 단계별로 배웁니다.

코드를 먼저 배운 뒤에 AI 를 쓰는 과정이 아닙니다. AI 에게 맡기면서 코드와 개발 원리를 함께 이해하는 과정입니다. 그래서 각 단계는 "하네스의 한 요소"를 하나씩 익히도록 설계했습니다.

대상	프로그래밍을 처음 배우는 사람 (코딩 경험이 없어도 됩니다)
준비물	Windows PC, Git, OpenCode, OpenRouter 계정 (1단계부터 Python 추가)
필요 없는 것	Docker, 데이터베이스, 웹 서버 (0·1단계 기준)
에이전트	OpenCode (OpenRouter 모델 사용)

단계 안내

단계	대상	내용	상태
0단계	코드를 전혀 모르는 사람	프로그래밍 언어 없이, HTML·CSS 명함 페이지 한 장을 AI 에게 맡기고 하네스의 원리를 익힌다	**이 저장소에 있음** (`docs/stage0/`)
1단계	생초보	Python 계산기와 할 일 목록 프로그램으로, 테스트와 저장 장치까지 익힌다	**이 저장소에 있음** (`docs/stage1/`)
2단계	초급	도서 대출 목록 프로그램으로, 저장, 상태 규칙, 요구사항 변경, 되돌리기를 익힌다	**이 저장소에 있음** (`docs/stage2/`)
3단계	실무형	3일 과정. FastAPI, PostgreSQL, Docker 까지 연결하는 전체 실습	별도 저장소 harness-crud-lab

2단계 교시표

장	주제	하네스 개념	결과물
준비하기	1단계 도구 확인, 작업 폴더 만들기	도구와 실행 환경	설치 확인
1장	도서관 요구사항 쓰기	생활 규칙을 상태로 바꾸기	도서관 요구사항
2장	파일에 저장하기	상태를 파일로 남기기	책 등록, JSON 저장
3장	대출과 반납	작은 지시로 나눠 맡기기, 상태 규칙	대출과 반납 기능
4장	AI 가 만든 실수 찾기	테스트가 잡는 실수와 못 잡는 실수	빠진 약속을 추가한 테스트
5장	요구사항이 바뀌었을 때	기준은 사람이, 구현은 AI 가	변경 요청서, 대출 기간 기록
6장	잘못된 변경을 되돌리기	`restore` 와 `revert`	되돌린 기록
7장	미니 프로젝트: 도서 대출 목록	1~6장 종합	완성된 프로그램, 과정 기록

도커로 실행하기 (선택)

PC 에 Python 과 OpenCode 를 직접 설치하지 않고, 컨테이너 안에서 0·1·2단계를 할 수 있습니다. 방법은 `docs/docker/00_도커로_실행하기.md` 를 봅니다. 편집은 Windows 의 VS Code 에서, 명령은 컨테이너 안에서 합니다.

0단계 교시표

장	주제	결과물
0장	준비하기 (Git, OpenCode, OpenRouter 연결)	설치 확인
1장	AI 와 하네스 체험	조건 없이 시킨 것과 조건을 붙인 것의 비교
2장	말로 된 요구사항 쓰기	명함 요구사항
3장	작업 지시서 쓰기	여섯 칸 지시서
4장	규칙과 권한 세우기	`AGENTS.md`, 권한 설정 확인
5장	체크리스트로 확인하고 저장하기	체크리스트 확인 후 커밋
6장	미니 프로젝트: 자기소개 명함 페이지	HTML·CSS 명함 페이지, 과정 기록

1단계 교시표

교시	주제	배우는 것	결과물
0장	준비하기	Python, Git, OpenCode, OpenRouter 연결	설치 확인 체크리스트
1장	하네스란 무엇인가	같은 부탁을 조건 없이 한 것과 조건을 붙인 것의 차이	비교 기록
2장	요구사항 쓰기	"이렇게 하면 → 이런 결과"로 흐리지 않게 쓰기	계산기 요구사항
3장	작업 지시서 쓰기	목표·범위·하지 말 것·완료 조건 여섯 칸	지시서와 계획 검토
4장	규칙과 권한 세우기	`AGENTS.md`(부탁)와 `opencode.jsonc`(잠금장치)	규칙 문서, 권한 확인
5장	검증하고 저장하기	빨강→초록, 검증을 통과해야 커밋되는 훅	통과한 커밋
6장	미니 프로젝트	할 일 목록을 AI 에게 맡기고 통제하기	할 일 목록 프로그램, 과정 기록

각 장은 **학습 목표 → 개념 → 따라 하기 → 확인하기 → 자주 만나는 오류 → 확인 문제** 순서입니다.

저장소 구조

```
harness-beginner-tutorial/
├─ README.md                이 문서
├─ docs/
│   ├─ stage0/              0단계 교안 (0장~6장)
│   ├─ stage1/              1단계 교안 (0장~6장)
│   ├─ docker/              도커로 실행하기 (0·1·2단계 공통)
│   ├─ stage2/              2단계 교안 (준비하기, 1장~7장)
│   ├─ 하네스_입문_0단계_교안.pdf
│   ├─ 하네스_입문_1단계_교안.pdf
│   └─ 하네스_입문_2단계_교안.pdf
├─ templates/               수업에서 복사해서 쓰는 틀과 파일
│   ├─ stage0/              0단계 틀 (명함 요구사항·지시서·규칙·권한·체크리스트)
│   ├─ stage2/              2단계 틀 (도서관 요구사항·지시서·변경 요청서·테스트 기준)
│   ├─ REQ_빈칸.md           2장: 요구사항 빈칸 틀
│   ├─ TASK_빈칸.md          3장: 작업 지시서 빈칸 틀
│   ├─ AGENTS_빈칸.md        4장: 규칙 문서 빈칸 틀
│   ├─ opencode.jsonc       4장: 권한 설정 (완성본)
│   ├─ REQUIREMENTS.md      6장: 할 일 목록 요구사항
│   ├─ TASK_todo.md         6장: 할 일 목록 작업 지시서
│   ├─ tests/test_todo.py   6장: 할 일 목록 테스트 (AI 가 고칠 수 없는 기준)
│   ├─ verify.py            5장: 검증 스크립트
│   ├─ pre-commit           5장: 커밋 전 검증 훅
│   └─ .gitattributes       줄바꿈 설정
├─ docker/                  도커 실행 환경 (0·1·2단계 공용)
│   ├─ Dockerfile           Python, pytest, Git, OpenCode 가 든 이미지 설계도
│   ├─ docker-compose.yml   작업 폴더와 템플릿 폴더를 연결하는 설정
│   └─ .env.example         경로와 Git 이름 설정 견본
├─ examples/
│   ├─ calc/                1단계 4·5장에서 쓰는 계산기 예제 (multiply 가 빠진 상태)
│   └─ stage2/              2단계 4장에서 쓰는 연습용 초안 (검증 전, 실수 포함)
└─ tools/
    └─ build_guide.py       교안 PDF 를 만드는 스크립트 (인자: 0 또는 1)
```

시작하기

0단계(`docs/stage0/`), 1단계(`docs/stage1/`), 2단계(`docs/stage2/`) 순서로 따라 합니다.

■ **[PS] 윈도우 PowerShell** 저장소를 내려받습니다.

```
git clone https://github.com/sinaboro/harness-beginner-tutorial C:\harness-beginner-tutorial
```

교안은 PDF 로도 볼 수 있습니다. `docs/하네스_입문_0단계_교안.pdf`, `docs/하네스_입문_1단계_교안.pdf`, `docs/하네스_입문_2단계_교안.pdf` 를 여세요.

0단계부터 순서대로 합니다. 0단계는 Python 이 필요 없고, 1단계부터 Python 을 씁니다.

원칙 세 가지

1. **AI 가 쓰고, 사람이 판단합니다.** 코드는 AI 가 쓰더라도, 계획을 승인하고 결과를 읽고 저장하는 판단은 사람이 합니다.
2. **기준은 AI 가 고치지 않습니다.** 테스트와 검증 스크립트, 규칙 문서는 권한 설정으로 AI 가 고치지 못하게 막습니다.
3. **통과한 것만 저장합니다.** 검증이 실패하면 커밋 자체가 취소됩니다.

강사 안내

- 교안의 실행 결과는 0장~6장의 명령을 실제 Windows PowerShell 에서 확인한 것을 기준으로 합니다.
  OpenCode 의 AI 답변은 실행할 때마다 다르므로, 교안에는 답변 내용 대신 **확인 방법**을 적었습니다.
- 6장에는 정답 코드를 싣지 않았습니다. 학생이 테스트와 지시서를 읽고 AI 와 대화하며 맞춰 가도록 한 것입니다.
- 3단계(3일 과정)는 별도 저장소 harness-crud-lab 에 있습니다. 2단계 7장을 마치면 그 저장소로 이어 갑니다.

2단계 준비하기 (도커)

시간	30분
준비물	1단계를 마친 상태 (도커 이미지와 OpenRouter 연결이 있다)
결과	컨테이너 안에 도서 대출 목록 작업 폴더가 만들어지고, 1단계 도구가 그대로 동작한다

2단계는 1단계와 **같은 도커 이미지**를 씁니다. 새로 설치할 것은 없습니다.

> **역할 나누기**
> - **Windows 쪽(PowerShell)**: 컨테이너를 켠다. `[Windows]` 표시.
> - **컨테이너 쪽(bash)**: 작업 폴더를 만들고 실습한다. `[컨테이너]` 표시.

1. 컨테이너 켜기 (Windows)

[Windows] (PowerShell)

```
cd C:\harness-beginner-tutorial\docker
docker compose run --rm workspace
```

2. 도구 확인 (컨테이너 안)

[컨테이너]

```
python --version
python -m pytest --version
git --version
opencode --version
```

네 줄 모두 버전이 나와야 합니다. OpenCode 연결은 `opencode` → `/models` 로 확인하고 `/exit` 로 나옵니다.

3. 작업 폴더 만들기 (컨테이너 안)

[컨테이너]

```
mkdir -p /workspace/library/tests
cd /workspace/library
git init
git config core.hooksPath .githooks
```

> 이 폴더 `/workspace/library` 는 Windows 의 `C:\harness-lab\library` 입니다. 2장부터 7장까지 모두 여기서 합니다.

4. 1단계 도구 복사하기 (컨테이너 안)

[컨테이너]

```
T="/tutorial/templates"
cp "$T/verify.py" .
cp "/tutorial/templates/stage2/pytest.ini" .
cp "$T/opencode.jsonc" .
cp /workspace/calc/AGENTS.md AGENTS.md
sed -i 's/calc\.py/library.py/g' AGENTS.md
cp "$T/.gitattributes" .
```

```
mkdir -p .githooks
cp "$T/pre-commit" .githooks/pre-commit
echo ".env" > .gitignore
ls -la
```

- `AGENTS.md` 는 1단계 4장에서 채운 파일(`/workspace/calc/AGENTS.md`)을 가져온 뒤, `sed` 명령으로 `calc.py` 를 `library.py` 로 바꿉니다. 1단계 4장을 건너뛰었다면 먼저 1단계 4장을 마칩니다.

확인 체크리스트

- ■ 도구 네 가지 버전이 나온다
- ■ `/workspace/library` 에 `verify.py`, `pytest.ini`, `opencode.jsonc`, `AGENTS.md`, `.githooks` 가 있다
- ■ `git status` 가 실행된다

2단계 1장. 도서관 요구사항 쓰기

> **실행 환경: 도커.** 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간
준비물	준비하기 완료
결과	도서 대출 목록의 요구사항을 "이렇게 하면 → 이런 결과 → 잘못 쓰면"으로 쓴다

학습 목표

1. 생활 속 규칙("이미 빌려 간 책은 못 빌린다")을 요구사항으로 옮긴다.
2. 상태(대출 가능, 대출 중)를 명확한 말로 정한다.
3. "잘못 쓰면" 칸을 채워 빈틈을 찾는다.

1. 개념 — 규칙은 상태에서 나온다

도서관에는 책 한 권마다 **상태**가 있습니다. 대출 가능, 대출 중입니다. 대부분의 규칙은 상태에서 나옵니다.

생활 속 규칙	상태로 바꾼 말	요구사항
빌려 간 책은 다른 사람이 못 빌린다	대출 중이면 대출할 수 없다	대출 중인 책을 빌리려 하면 오류
빌리지 않은 책은 반납할 수 없다	대출 가능이면 반납할 수 없다	대출 가능한 책을 반납하려 하면 오류
빌리는 사람 이름이 있어야 한다	이름이 비면 대출할 수 없다	이름이 공백이면 오류

> 1단계 2장의 "잘못 쓰면" 칸이 여기서 가장 중요합니다. 규칙마다 **잘못된 경우**가 하나씩 있습니다.

2. 따라 하기

1단계. 생활 규칙 찾기 (15분)

도서관에 가면 볼 수 있는 규칙을 다섯 개 적습니다. 예: "책은 한 번에 한 권만 빌린다", "반납 기한이 있다".

생활 속 규칙	이 프로그램에서 다룰까? (예/아니오)	아니오면 이유

2단계. 요구사항 틀 채우기 (40분)

1단계 틀 `templates\REQ_빈칸.md` 를 복사합니다.

[컨테이너]

```
mkdir -p /workspace/library-req
cp /tutorial/templates/REQ_빈칸.md /workspace/library-req/요구사항.md
# (Windows) /workspace/library-req/요구사항.md 을(를) VS Code 로 열어 고칩니다
```

다음 기능을 채웁니다.

- **R1. 책 등록** (제목, 지은이)
- **R2. 목록 보기** (전체, 대출 가능, 대출 중)

- **R3. 대출** (번호와 빌리는 사람 이름)
- **R4. 반납** (번호)
- **R5. 파일 저장과 읽기**

R3, R4 에는 반드시 "잘못 쓰면" 칸을 채웁니다.

- R3: 없는 번호, 이미 대출 중, 이름이 공백
- R4: 없는 번호, 대출 중이 아닌 책

3단계. 빈칸 점검하기 (20분)

아래 질문으로 점검합니다.

- ■ "책 목록" 같은 흐린 말이 있나? 어떤 목록인지 상태로 말했나?
- ■ 반납하면 빌린 사람 이름은 어떻게 되나? 요구사항에 적혀 있나?
- ■ 책 번호는 어떻게 정해지나? 지우는 기능이 있으면 번호는 어떻게 되나?
- ■ "만들지 않는 것"을 적었나? (연체, 날짜 계산 등)

> 세 번째 질문이 함정입니다. 4교시와 5교시에서 이 빈틈이 다시 나옵니다.

4단계. 옆 사람과 바꿔 읽기 (25분)

옆 사람의 요구사항에서 **"반납하면 빌린 사람 이름은 어떻게 되나요?"** 같은 질문을 하나씩 찾아 적습니다. 대답이 없으면 그 요구사항은 빈틈이 있는 것입니다.

3. 확인하기

- ■ 상태(대출 가능, 대출 중)가 요구사항에 명확히 적혀 있다.
- ■ R3, R4 의 "잘못 쓰면" 칸이 모두 채워져 있다.
- ■ 반납 시 빌린 사람 이름이 어떻게 되는지 요구사항에 적혀 있다.

4. 자주 만나는 오류

증상	해결
"관리를 잘한다" 같은 흐린 말이 나온다	상태로 바꾼다. 예: "대출 중인 책 목록을 보여 준다"
반납 후 상태를 빠뜨렸다	반납 전과 후의 상태를 한 줄씩 적는다.

5. 확인 문제

1. 대출 중인 책을 다시 빌리려 하면 어떤 결과가 나와야 하나? 요구사항에서 찾아 적으시오.
2. 반납하면 `borrower`(빌린 사람)는 무엇이 되어야 하나?
3. 요구사항에 "연체"를 넣지 않은 이유를 "만들지 않는 것"과 연결해서 설명하시오.

2단계 2장. 책 목록을 파일에 저장하기

> **실행 환경: 도커.** 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간
준비물	1장의 요구사항.md, 준비하기를 마친 /workspace/library
결과	책 등록, 목록, 파일 저장을 AI 에게 맡기고, 테스트와 대조한 뒤 저장한다

이 장에서 할 일 한눈에 보기

1. 테스트 기준 파일을 가져오고 읽는다. (15분)
2. 빨강을 확인한다. (10분)
3. AI 에게 지시서를 건네고 계획을 검토한다. (25분)
4. 초록을 확인하고 테스트 파일이 바뀌지 않았는지 대조한다. (15분)
5. 저장된 파일을 눈으로 확인한다. (15분)
6. 저장하기. (10분)

1단계. 테스트 기준 파일 가져와 읽기 (15분)

[컨테이너]

```
cd /workspace/library
cp /tutorial/templates/stage2/tests/test_books.py tests/
cp /tutorial/templates/stage2/TASK_책저장.md .
ls tests
```

[Windows] VS Code 에서 `C:\harness-lab\library\tests\test_books.py` 를 엽니다. 아래 세 질문에 **찾은 `assert` 줄**을 옮겨 적습니다.

질문	답 (찾은 줄을 그대로 옮긴다)
1. `add_book` 이 돌려주는 책의 모양은? (`test_add_book_creates_available_book` 의 `assert book ==` 줄)	
2. 공백뿐인 제목은 어떻게 되어야 하나? (`test_add_book_rejects_blank_title` 의 `with pytest.raises` 줄)	
3. 파일이 없으면 `load_books` 는 무엇을 돌려주나? (`test_load_missing_file_returns_empty_list` 의 `assert` 줄)	

2단계. 빨강 확인하기 (10분)

[컨테이너]

```
cd /workspace/library
python -m pytest tests/test_books.py
```

확인 기준: `library.py` 가 없으므로 실패합니다. 출력에서 `ModuleNotFoundError` 또는 `ImportError` 줄을 찾아 아래에 옮깁니다.

```
(옮겨 적을 줄)
```

3단계. AI 에게 지시서 건네고 계획 검토하기 (25분)

[컨테이너]

```
cd /workspace/library
opencode
```

■ **[AI] OpenCode 입력 칸**

```
TASK_책저장.md 를 읽고 그 지시대로 작업해 줘. 먼저 계획만 말하고 내 승인을 기다려.
```

계획을 아래 네 질문으로 검토합니다.

질문	답 (예/아니오)	아니오면 입력할 문장
1. 고치는 파일이 library.py 한 개뿐인가?		아래 문장 A
2. 계획에 lend_book, return_book 이 없는가? (이번 장에서는 만들지 않는다)		아래 문장 A
3. 계획에 tests/ 를 고친다는 말이 없는가?		아래 문장 A
4. 계획에 데이터베이스, 웹 화면, 외부 패키지가 없는가?		아래 문장 A

[AI] 문장 A (아니오일 때, 번호만 바꿔 입력합니다)

```
계획 [번호]번이 TASK_책저장.md 와 맞지 않아. 그 부분을 빼고 계획을 다시 말해 줘. 아직 파일은 고치지 마.
```

모두 예이면 입력합니다.

■ **[AI] OpenCode 입력 칸**

```
진행
```

끝나면 `/exit` 를 칩니다.

4단계. 초록 확인하고 테스트 파일 대조하기 (15분)

[컨테이너]

```
cd /workspace/library
python -m pytest tests/test_books.py
diff tests/test_books.py /tutorial/templates/stage2/tests/test_books.py && echo "테스트 파일은 원본과 같습니다"
```

확인 기준:

확인할 것	기대하는 결과	실제 결과 (예/아니오)
테스트 결과	`6 passed` 가 나온다	
테스트 파일	"테스트 파일은 원본과 같습니다"가 나온다	

아니오가 있으면 AI 에게 실패 줄을 알리고 `library.py` 만 고치게 합니다. 테스트 파일이 바뀌었다면 `cp /tutorial/templates/stage2/tests/test_books.py tests/` 로 원본을 다시 복사합니다.

5단계. 저장된 파일 눈으로 확인하기 (15분)

[컨테이너]

```
cd /workspace/library
python -c "
from library import add_book, save_books, load_books
books = []
add_book(books, '데미안', '헤세')
add_book(books, '노인과 바다', '헤밍웨이')
save_books(books, 'books.json')
print('저장 완료:', len(load_books('books.json')), '권')
"
cat books.json
```

확인 기준:

- `저장 완료: 2 권` 이 나와야 합니다.
- `cat books.json` 에 `"title": "데미안"` 처럼 한글이 **깨지지 않고** 보여야 합니다. `\uXXXX` 처럼 보이면 `ensure_ascii=False` 가 빠진 것입니다.

[Windows] VS Code 에서 `C:\harness-lab\library\books.json` 을 열어 같은 내용인지 봅니다.

6단계. 저장하기 (10분)

책 데이터 파일은 개인 기록이므로 저장소에 올리지 않습니다.

[컨테이너]

```
cd /workspace/library
echo "books.json" >> .gitignore
cat .gitignore
python verify.py
git add -A
git status
git commit -m "2장: 책 등록, 목록, JSON 저장 (R1, R2, R5)"
git log --oneline
```

확인 기준:

- `.gitignore` 에 `books.json` 줄이 있어야 합니다.
- `git status` 에 `books.json` 이 없어야 합니다.
- `git log --oneline` 에 커밋이 한 줄 생겨야 합니다.

확인 체크리스트

- ■ 테스트 기준 세 질문에 찾은 줄을 옮겨 적었다.
- ■ 빨강(ModuleNotFoundError 또는 ImportError)을 옮겨 적었다.
- ■ 계획 검토 네 질문에 답했다.
- ■ `6 passed` 와 테스트 파일 대조를 확인했다.
- ■ `books.json` 의 한글이 깨지지 않았다.
- ■ 커밋이 한 줄 생겼다.

자주 만나는 오류

증상	해결
`ModuleNotFoundError: No module named 'library'` (4단계 이후)	AI 가 `library.py` 를 만들지 않았다. `ls` 로 확인하고 AI 에게 다시 말한다.
`JSONDecodeError`	`books.json` 을 손으로 고쳐서 형식이 깨졌다. 파일을 지우고 5단계를 다시 한다.
한글이 `\uXXXX` 로 보인다	AI 에게 "save_books 에 ensure_ascii=False 를 넣어 줘"라고 말한다.
`git status` 에 `books.json` 이 보인다	6단계의 `.gitignore` 줄이 빠졌다. 다시 `echo "books.json" >> .gitignore` 를 친다.

2단계 3장. 대출과 반납: 작은 지시로 나눠 맡기기

실행 환경: 도커. 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간
준비물	2장을 마친 상태 (`library.py` 에 책 등록과 저장이 있고 커밋되어 있다)
결과	대출과 반납을 AI 에게 맡기고, 상태 규칙을 테스트와 직접 실행으로 확인한 뒤 저장한다

이 장에서 할 일 한눈에 보기

1. 상태 규칙 테스트를 가져와 읽는다. (15분)
2. 빨강을 확인한다. (10분)
3. AI 에게 지시서를 건네고 계획을 검토한다. (25분)
4. 초록을 확인하고 테스트 파일을 대조한다. (15분)
5. 상태 규칙을 직접 실행해서 확인한다. (20분)
6. 저장하기. (10분)

1단계. 상태 규칙 테스트 가져와 읽기 (15분)

[컨테이너]

```
cd /workspace/library
cp /tutorial/templates/stage2/tests/test_lending.py tests/
cp /tutorial/templates/stage2/TASK_대출반납.md .
ls tests
```

[Windows] VS Code 에서 `C:\harness-lab\library\tests\test_lending.py` 를 엽니다. 테스트 이름을 아래 표에 옮기고, 각 테스트가 무엇을 확인하는지 한 문장으로 씁니다.

테스트 이름 (그대로 옮긴다)	무엇을 확인하나 (한 문장)
test_lend_book_marks_lent	
test_lend_lent_book_raises	
test_lend_missing_book_raises	
test_lend_blank_borrower_raises	
test_return_book_marks_available	
test_return_available_book_raises	
test_return_missing_book_raises	

확인 기준: 표의 일곱 줄이 모두 채워져야 합니다. 컨테이너에서 `grep -c "^def test_" tests/test_lending.py` 를 치면 결과가 `7` 입니다.

2단계. 빨강 확인하기 (10분)

[컨테이너]

```
cd /workspace/library
python -m pytest tests/test_lending.py
```

확인 기준: 실패합니다. `AttributeError` 또는 `ImportError` 줄을 찾아 옮겨 적습니다.

```
(옮겨 적을 줄)
```

3단계. AI 에게 지시서 건네고 계획 검토하기 (25분)

[컨테이너]

```
cd /workspace/library
opencode
```

■ [AI] OpenCode 입력 칸

```
TASK_대출반납.md 를 읽고 그 지시대로 작업해 줘. 먼저 계획만 말하고 내 승인을 기다려.
```

계획을 아래 네 질문으로 검토합니다.

질문	답 (예/아니오)	아니오면 입력할 문장
1. 연체 계산이나 날짜 계산이 계획에 없는가?		아래 문장 A
2. 계획에 lend_book 과 return_book 이 모두 있는가?		아래 문장 A
3. 계획에 tests/ 를 고친다는 말이 없는가?		아래 문장 A
4. 계획이 library.py 한 개만 고치겠다고 하는가?		아래 문장 A

[AI] 문장 A (아니오일 때, 번호만 바꿔 입력합니다)

```
계획 [번호]번이 TASK_대출반납.md 와 맞지 않아. 그 부분을 빼고 계획을 다시 말해 줘. 아직 파일은 고치지 마.
```

모두 예이면 입력합니다.

■ [AI] OpenCode 입력 칸

```
진행
```

끝나면 `/exit` 를 칩니다.

4단계. 초록 확인하고 테스트 파일 대조하기 (15분)

[컨테이너]

```
cd /workspace/library
python -m pytest tests/test_books.py tests/test_lending.py
diff tests/test_lending.py /tutorial/templates/stage2/tests/test_lending.py && echo "테스트 파일은 원본과 같습니다"
```

확인 기준:

확인할 것	기대하는 결과	실제 결과 (예/아니오)
테스트 결과	`13 passed` 가 나온다	
테스트 파일	"테스트 파일은 원본과 같습니다"가 나온다	

아니오가 있으면 AI 에게 실패 줄을 알리고 `library.py` 만 고치게 합니다. 테스트 파일이 바뀌었다면 원본을 다시 복사합니다.

5단계. 상태 규칙을 직접 실행해서 확인하기 (20분)

테스트가 통과해도, 규칙이 눈에 보이는지 직접 확인합니다.

[컨테이너]

```
cd /workspace/library
python -c "
from library import add_book, lend_book, return_book
books = []
add_book(books, '데미안', '헤세')
print('1. 처음 상태:', books[0]['status'], books[0]['borrower'])
lend_book(books, 1, '철수')
print('2. 대출 후:', books[0]['status'], books[0]['borrower'])
try:
    lend_book(books, 1, '영희')
    print('3. 다시 빌림: 오류가 없었다 (잘못됨)')
except ValueError as e:
    print('3. 다시 빌림: 오류 =', e)
return_book(books, 1)
print('4. 반납 후 상태:', books[0]['status'])
"
```

확인 기준:

줄	기대하는 결과	실제 출력 (그대로 옮긴다)	맞는가? (예/아니오)
1	처음 상태: available None		
2	대출 후: lent 철수		
3	다시 빌림: 오류 = (오류 문장)		
4	반납 후 상태: available		

오류 문장의 글자는 AI 가 만든 것에 따라 달라도 됩니다. 중요한 것은 3번 줄에 "오류 ="가 있는 것입니다.
아니오가 있으면 AI 에게 그 줄 번호를 알리고 고치게 합니다.

6단계. 저장하기 (10분)

[컨테이너]

```
cd /workspace/library
python verify.py
git add -A
git status
git commit -m "3장: 대출과 반납, 상태 규칙 (R3, R4)"
git log --oneline
```

확인 기준: `git status` 에 `books.json` 이 없어야 합니다(`.gitignore` 에 있다). 커밋이 한 줄 생기면 끝입니다.

확인 체크리스트

- ■ 테스트 이름 일곱 개와 각 확인 내용을 적었다.
- ■ 빨강을 옮겨 적었다.
- ■ 계획 검토 네 질문에 답했다.
- ■ `13 passed` 와 테스트 파일 대조를 확인했다.

- ■ 직접 실행 네 줄의 결과를 적었다.
- ■ 커밋이 한 줄 생겼다.

자주 만나는 오류

증상	해결
`AttributeError: module 'library' has no attribute 'lend_book'`	AI 가 `lend_book` 을 만들지 않았다. AI 에게 함수 이름을 알리고 추가하게 한다.
직접 실행 3번 줄에 "오류가 없었다"가 나온다	이미 대출 중인 책을 막는 검사가 없다. AI 에게 "이미 대출 중이면 ValueError 를 일으키게 해 줘"라고 말한다.
`13 passed` 가 아니다	실패 줄을 옮겨 적고, AI 에게 원인부터 설명하게 한다.

2단계 4장. AI 가 만든 실수 찾기

> **실행 환경: 도커.** 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간
준비물	3장을 마친 상태 (`library.py` 가 커밋되어 있다)
결과	AI 가 만든 초안의 실수를 테스트로 잡고, 테스트가 못 잡는 실수를 직접 찾아 테스트를 추가한다

이 장에서 할 일 한눈에 보기

1. 내 완성본을 옆에 보관하고, AI 초안(실수 있음)으로 바꾼다. (10분)
2. 테스트로 실수를 찾는다. (15분)
3. AI 에게 고치게 한다. (20분)
4. 테스트가 못 잡는 실수를 직접 찾는다. (20분)
5. 빠진 테스트를 사람이 쓰고, AI 가 고치게 한다. (25분)
6. 비교하고 저장한다. (15분)

1단계. 내 완성본 보관하고 초안으로 바꾸기 (10분)

[컨테이너]

```
cd /workspace/library
cp library.py library_mine.py
cp /tutorial/examples/stage2/library_bugged.py library.py
ls
```

확인 기준: `library_mine.py` 와 `library.py` 가 모두 보입니다. `library.py` 는 AI 가 만든 것처럼 준비된 초안입니다.

> `library_mine.py` 는 6단계에서 비교한 뒤 지웁니다.

2단계. 테스트로 실수 찾기 (15분)

[컨테이너]

```
cd /workspace/library
python -m pytest tests/test_books.py tests/test_lending.py
```

출력의 마지막 줄을 보고 아래 표를 채웁니다. 실패 줄은 `FAILED` 로 시작합니다.

실패한 테스트 이름 (그대로 옮긴다)	이 테스트가 말하는 약속 (한 문장. 테스트 함수 이름과 `assert` 줄을 보고 쓴다)

확인 기준: 실패가 **두 개**여야 합니다. (`2 failed, 11 passed`) 다른 개수라면 1단계를 다시 합니다.

3단계. AI 에게 고치게 하기 (20분)

[컨테이너]

```
cd /workspace/library
opencode
```

■ [AI] OpenCode 입력 칸

```
python -m pytest tests/test_books.py tests/test_lending.py 를 실행하면 실패 두 개가 나와. 실패한 테스트 이름과 아래 출력을 보고, 원인을
  먼저 설명해 줘.
(실패 출력의 마지막 15줄을 붙여 넣는다)
내가 "진행"이라고 하면 library.py 만 고쳐 줘. tests/ 는 건드리지 마.
```

AI 의 원인 설명을 읽고 아래 두 질문에 답합니다.

질문	답 (예/아니오)
1. 설명한 원인이 2단계 표의 "약속"과 같은 내용인가?	
2. 계획에 tests/ 를 고친다는 말이 없는가?	

모두 예이면 입력합니다.

■ [AI] OpenCode 입력 칸

```
진행
```

끝나면 `/exit` 를 칩니다. 다시 확인합니다.

[컨테이너]

```
cd /workspace/library
python -m pytest tests/test_books.py tests/test_lending.py
```

확인 기준: `13 passed` 가 나와야 합니다. 이 시점에 테스트는 모두 통과하지만, 실수가 아직 하나 남아 있습니다.

4단계. 테스트가 못 잡은 실수 직접 찾기 (20분)

테스트가 모두 통과한 뒤에도 실수가 남았는지 직접 확인합니다.

[컨테이너]

```
cd /workspace/library
python -c "
from library import add_book, lend_book, return_book
books = []
add_book(books, '데미안', '헤세')
lend_book(books, 1, '철수')
print('반납 전:', books[0])
print('반납 후:', return_book(books, 1))
"
```

확인 기준: 반납 후 출력에서 `'status'` 와 `'borrower'` 를 찾아 옮깁니다.

확인할 것	실제 값 (출력에서 옮긴다)	약속대로인가? (예/아니오)
반납 후 `status`		
반납 후 `borrower`		

약속: 반납하면 빌린 사람 이름(`borrower`)이 `None` 이 되어야 합니다. (REQUIREMENTS 의 R4 와 같습니다.)

`borrower` 가 `None` 이 아니면 **테스트가 못 잡은 실수**입니다. 3장의 테스트는 `status` 만 확인했기 때문입니다.

5단계. 빠진 테스트를 사람이 쓰고, AI 에게 고치게 하기 (25분)

무엇을: 테스트 파일에 약속을 한 줄 더합니다. **이것은 사람이 씁니다.** AI 에게 맡기지 않습니다.

[Windows] VS Code 에서 `C:\harness-lab\library\tests\test_lending.py` 를 엽니다. 파일 맨 아래에 아래 블록을 **그대로** 붙여 넣고 저장합니다. 앞에 빈 줄 두 개를 둡니다.

```


def test_return_clears_borrower():
    # 반납하면 빌린 사람 이름을 지운다. (요구사항 R4)
    books = []
    add_book(books, "데미안", "헤세")
    lend_book(books, 1, "철수")
    book = return_book(books, 1)
    assert book["borrower"] is None
```

[컨테이너]

```
cd /workspace/library
python -m pytest tests/test_lending.py::test_return_clears_borrower
```

확인 기준: **실패해야 합니다.** (빨강) 실패하지 않으면 블록을 다시 읽고, 파일 맨 아래에 붙었는지 확인합니다.

그다음 AI 에게 고치게 합니다.

■ [AI] OpenCode 입력 칸

```
tests/test_lending.py 의 test_return_clears_borrower 가 실패해. 반납할 때 borrower 를 None 으로 바꾸도록 library.py 의 return_book 을 고쳐 줘. 계획을 먼저 말하고 내가 "진행"이라고 하면 library.py 만 고쳐. tests/ 는 건드리지 마.
```

계획을 확인하고 `진행` 을 입력합니다. 끝나면 `/exit` 를 칩니다.

[컨테이너]

```
cd /workspace/library
python verify.py
```

확인 기준: "검증 통과"가 나와야 합니다. (초록)

6단계. 내 완성본과 비교하고 저장하기 (15분)

[컨테이너]

```
cd /workspace/library
diff library_mine.py library.py
```

확인 기준: 차이가 나는 줄을 두 개 이상 찾아 적습니다. 특히 `return_book` 함수의 차이를 한 문장으로 적습니다.

```
return_book 의 차이: (여기에 한 문장)
```

비교가 끝나면 `library_mine.py` 를 지우고 저장합니다.

[컨테이너]

```
cd /workspace/library
rm -f library_mine.py
python verify.py
git add -A
git status
git commit -m "4장: 테스트가 못 잡은 반납 실수를 테스트로 추가하고 고침 (R4)"
git log --oneline
```

확인 기준: 커밋이 한 줄 생기고, `git status` 에 `library_mine.py` 가 없어야 합니다.

확인 체크리스트

- ■ 실패가 두 개였고, 각 실패의 약속을 한 문장으로 적었다.
- ■ 테스트가 모두 통과한 뒤에도 borrower 가 남는 것을 직접 봤다.
- ■ 빠진 테스트를 사람이 쓰고, 실패하는 것을 확인했다.
- ■ AI 가 `library.py` 만 고쳤고, `verify.py` 가 통과한다.
- ■ 차이를 한 문장으로 적었고, 비교용 파일을 지웠다.

자주 만나는 오류

증상	해결
실패가 두 개가 아니다	`cp /tutorial/examples/stage2/library_bugged.py library.py` 를 다시 친다.
새 테스트가 처음부터 통과한다	붙여 넣은 함수 이름이 `test_` 로 시작하는지 본다. 블록이 파일 맨 아래에 있는지 본다.
AI 가 테스트를 고쳐서 통과시켰다	`diff tests/test_lending.py /tutorial/templates/stage2/tests/test_lending.py` 로 확인한다. 5단계에서 추가한 테스트 때문에 달라진 것은 정상이다. 그 밖의 줄이 다르면 AI 에게 되돌리게 한다.
`library_mine.py` 를 지우는 것을 잊었다	`rm -f library_mine.py` 를 친 뒤 커밋한다.

2단계 5장. 요구사항이 바뀌었을 때

> **실행 환경: 도커.** 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간
준비물	4장까지 완료. 템플릿 `/tutorial/templates/stage2/CHANGE_요청서.md`, `/tutorial/templates/stage2/tests/test_lending_v2.py`
결과	변경 요청서를 칸 채우기로 쓰고, 기준(테스트)은 사람이 바꾼 뒤 AI 에게 구현시킨다

이 장에서 할 일 한눈에 보기

1. 변경 전 상태를 확인한다. (10분)
2. 변경 요청서를 칸별로 채운다. (25분)
3. 영향 받는 테스트를 찾는다. (10분)
4. 승인하고, 기준을 사람이 바꾼다. (20분)
5. 바뀐 기준으로 빨강을 확인한다. (10분)
6. AI 에게 구현시키고 대조한다. (30분)
7. 저장한다. (15분)

1단계. 변경 전 상태 확인하기 (10분)

[컨테이너]

```
cd /workspace/library
python verify.py
git status
```

확인 기준: "검증 통과"가 나오고, `git status` 에 바뀐 파일이 없어야 합니다("nothing to commit" 또는 `books.json` 만 무시됨). 이 상태가 **변경 전 기준점**입니다.

2단계. 변경 요청서 칸별로 채우기 (25분)

[컨테이너]

```
cd /workspace/library
cp /tutorial/templates/stage2/CHANGE_요청서.md .
```

[Windows] VS Code 에서 `C:\harness-lab\library\CHANGE_요청서.md` 를 엽니다. 아래 표의 칸을 채웁니다. 칸마다 **그대로 쓸 문장**이 있는 곳은 그 문장을 쓰고, 빈칸은 내 말로 씁니다.

요청서의 칸	어떻게 채우나
1. 요구사항 번호	`- 요구사항 번호: R3, R4` 는 그대로 둔다
1. 무엇이 바뀌나	`- 무엇이 바뀌나: 대출하면 loan_days 칸에 14 가 함께 기록된다. 반납하면 그 칸을 지운다.` 로 바꾼다
1. 왜 바꾸나	`- 왜 바꾸나: [빈칸]` 을 "도서관에서 대출 기간을 알아야 하기 때문이다" 또는 내 말로 한 문장 쓴다

요청서의 칸	어떻게 채우나
2. 영향을 받는 테스트	아래 3단계에서 찾은 테스트 이름을 적는다
3. 테스트 기준을 어떻게 바꾸나	그대로 둔다 (아래 4단계에서 사람이 한다)
4. 승인	승인 날짜와 이름은 4단계 뒤에 적는다

확인 기준: 요청서에 `[` 가 남아 있지 않아야 합니다. 컨테이너에서 `grep -c "\[" CHANGE_요청서.md` 를 치면 `0` 입니다. (4번 승인 줄의 빈칸은 `_____` 밑줄이라서 대괄호가 없어도 됩니다.)

3단계. 영향을 받는 테스트 찾기 (10분)

[컨테이너]

```
cd /workspace/library
grep -n "assert book ==" tests/test_lending.py
```

확인 기준: 책 모양 전체를 `==` 로 비교하는 줄이 나옵니다. 그 줄이 속한 **테스트 함수 이름**을 찾아 요청서 2번 칸에 적습니다.

```
grep -n "^def test_" tests/test_lending.py
```

줄 번호가 가장 가까운 `def test_` 줄이 함수 이름입니다.

확인 질문:

- ■ 이 변경으로 책 모양이 달라지는 테스트를 찾았다.
- ■ 책 상태만 확인하는 테스트(예: status 만 보는 것)는 적지 않았다.

4단계. 승인하고 테스트 기준을 사람이 바꾸기 (20분)

이 단계는 **사람이** 합니다. AI 에게 맡기지 않습니다.

(1) 승인: 요청서 4번에 승인 날짜와 이름을 적습니다.

(2) 새 기준을 옆에 두기:

[컨테이너]

```
cd /workspace/library
cp /tutorial/templates/stage2/tests/test_lending_v2.py tests/test_lending_v2.py
ls tests
```

(3) 4장에서 내가 추가한 테스트를 옮기기: 4장에서 `test_lending.py` 맨 아래에 붙인 `test_return_clears_borrower` 함수는 새 기준에 없습니다. 그래서 옮깁니다.

[Windows] VS Code 에서 `tests\test_lending.py` 를 열어 `def test_return_clears_borrower():` 부터 파일 끝까지를 복사합니다. 그다음 `tests\test_lending_v2.py` 맨 아래에 빈 줄 두 개를 두고 붙여 넣고 저장합니다.

(4) 새 기준으로 바꾸기:

[컨테이너]

```
cd /workspace/library
mv tests/test_lending_v2.py tests/test_lending.py
ls tests
```

확인 기준: `tests` 폴더에 `test_books.py` 와 `test_lending.py` 가 있고, `test_lending_v2.py` 는 없어야 합니다.

5단계. 바뀐 기준으로 빨강 확인하기 (10분)

[컨테이너]

```
cd /workspace/library
python -m pytest tests/test_lending.py
```

확인 기준:

- 실패가 **하나 이상** 나옵니다. 실패한 테스트 이름을 옮겨 적습니다.

```
(실패한 테스트 이름과 FAILED 줄)
```

- 실패한 테스트는 대출(lend)과 관련된 것이어야 합니다. 반납 테스트만 실패하면 4장의 반납 수정을 다시 확인합니다.

> 실패는 정상입니다. 기준이 바뀌었으니 구현이 아직 따라오지 않은 것입니다. 실패를 보고 "구현이 틀렸다"고 바로 결론 내리지 않습니다.

6단계. AI 에게 구현시키고 대조하기 (30분)

[컨테이너]

```
cd /workspace/library
opencode
```

■ [AI] OpenCode 입력 칸

```
CHANGE_요청서.md 의 변경을 구현해 줘. tests/ 는 이미 새 기준으로 바뀌어 있으니 고치지 마.
계획을 먼저 말하고, 내가 "진행"이라고 하면 library.py 만 고쳐 줘.
```

계획을 아래 세 질문으로 검토합니다.

질문	답 (예/아니오)	아니오면 입력할 문장
1. 고치는 파일이 library.py 한 개뿐인가?		아래 문장 A
2. 대출할 때 loan_days 를 14 로 넣고, 반납할 때 지운다고 했는가?		아래 문장 A
3. tests/ 를 고친다는 말이 없는가?		아래 문장 A

[AI] 문장 A (아니오일 때, 번호만 바꿔 입력합니다)

```
계획 [번호]번이 CHANGE_요청서.md 와 맞지 않아. 그 부분을 고쳐서 계획을 다시 말해 줘. 아직 파일은 고치지 마.
```

모두 예이면 입력합니다.

■ [AI] OpenCode 입력 칸

```
진행
```

끝나면 /exit 를 칩니다. 대조합니다.

[컨테이너]

```
cd /workspace/library
python -m pytest tests/test_books.py tests/test_lending.py
diff tests/test_lending.py /tutorial/templates/stage2/tests/test_lending_v2.py
```

확인 기준:

확인할 것	기대하는 결과	실제 결과 (예/아니오)
테스트 결과	모두 통과한다 (13 passed)	

확인할 것	기대하는 결과	실제 결과 (예/아니오)
diff 출력	원본 2차 기준과 다른 줄이 **4장에서 옮긴 테스트 블록 뿐**이다 (> 로 시작하는 줄만 보인다)	

diff 에서 < 로 시작하는 줄(기준에서 바뀐 줄)이 있으면 AI 가 테스트를 고친 것입니다. 되돌리게 합니다.

7단계. 저장하기 (15분)

[컨테이너]

```
cd /workspace/library
python verify.py
git add -A
git status
git commit -m "5장: 대출 기간 14일 기록 (요구사항 변경 승인됨, 기준은 사람이 변경, R3·R4)"
git log --oneline
```

확인 기준: 커밋 메시지에 "요구사항 변경 승인됨"이 들어가야 합니다. `git status` 에 `CHANGE_요청서.md` 가 보여도 함께 저장합니다.

확인 체크리스트

- ■ 변경 전 `python verify.py` 가 "검증 통과"였다.
- ■ 요청서 1~2번 칸을 채웠고, 대괄호가 남지 않았다.
- ■ 영향 받는 테스트 이름을 찾아 적었다.
- ■ 승인 날짜와 이름을 적었다.
- ■ 4장 테스트 블록을 새 기준에 옮겼다.
- ■ 바뀐 기준으로 실패를 확인했다.
- ■ 계획 검토 세 질문에 답했고, `diff` 결과를 확인했다.
- ■ 커밋 메시지에 승인 여부가 들어 있다.

자주 만나는 오류

증상	해결
`mv` 후에도 `test_lending_v2.py` 가 남아 있다	`ls tests` 로 확인한다. `mv` 가 실패했다면 파일 이름을 다시 쓴다.
4장 테스트 블록을 옮기지 않았다	기준 교체 후 `test_return_clears_borrower` 가 사라진다. 4단계 (3)을 다시 한다.
AI 가 `tests/test_lending.py` 를 고쳐서 통과시켰다	`diff` 의 < 줄을 찾는다. AI 에게 "tests 를 원래대로 되돌리고 library.py 만 고쳐 줘"라고 말한다.
실패가 반납 테스트에서만 난다	4장의 반납 수정(`borrower` 지우기)이 남아 있는지 `grep -n "borrower" library.py` 로 확인한다.

2단계 6장. 잘못된 변경을 되돌리기

> **실행 환경: 도커.** 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간
준비물	5장까지 완료 (커밋 기록이 여러 줄 있다)
결과	저장 전 변경은 `git restore` 로, 저장 후 변경은 `git revert` 로 되돌리고, 되돌린 뒤 검증한다

이 장에서 할 일 한눈에 보기

1. AI 에게 요구사항에 없는 기능을 시키고, 검증이 통과하는지 본다. (20분)
2. 범위를 대조해서 잘못을 찾는다. (15분)
3. 저장 전 변경을 `git restore` 로 되돌린다. (15분)
4. 같은 변경을 저장하고, `git revert` 로 되돌린다. (30분)
5. 되돌린 뒤 검증하고, 기록을 확인한다. (20분)

1단계. 요구사항에 없는 기능 시키기 (20분)

이 장은 **실수를 일부러 만드는 연습**입니다. 계획 검토를 건너뛰라고 말하는 것이 핵심입니다.

[컨테이너]

```
cd /workspace/library
git status
opencode
```

■ [AI] OpenCode 입력 칸

```
library.py 에 책 삭제 기능 delete_book(books, book_id) 를 추가해 줘. 계획 없이 바로 진행해 줘.
```

AI 가 끝나면 `/exit` 를 칩니다. 확인합니다.

[컨테이너]

```
cd /workspace/library
python verify.py
git status
grep -n "def delete_book" library.py
```

확인 기준:

- `python verify.py` 가 "검증 통과"로 끝납니다. (테스트는 깨지지 않았다)
- `grep` 이 `def delete_book` 줄을 찾습니다. (범위 밖 기능이 들어왔다)

2단계. 범위를 대조해서 잘못 찾기 (15분)

[Windows] VS Code 에서 `C:\harness-lab\library-req\요구사항.md` 를 엽니다. 1장에서 쓴 요구사항입니다. 이 파일이 이번 장의 기준입니다. 아래 표를 채웁니다.

질문	답 (예/아니오)
1. 요구사항에 "책 삭제" 기능이 있는가? (`/workspace/library-req/요구사항.md` 의 "만들지 않는 것" 줄을 본다)	
2. 검증이 통과했다는 것은 delete_book 이 맞다는 뜻인가?	

확인 기준:

- 1번이 "예"이면, 삭제 기능은 "만들지 않는 것"에 적혀 있다는 뜻입니다. 범위 밖입니다.
- 2번은 **아니오**여야 합니다. 검증은 "적힌 약속을 지켰다"만 확인합니다. 약속에 없는 기능은 확인하지 않습니다.

> **이 장의 첫 번째 교훈:** "검증 통과"와 "범위 안에 있다"는 다릅니다. 범위는 요구사항과 지시서로 따로 확인합니다.

3단계. 저장 전 변경을 git restore 로 되돌리기 (15분)

[컨테이너]

```
cd /workspace/library
git status
git restore library.py
grep -c "def delete_book" library.py
python verify.py
git status
```

확인 기준:

- `git restore library.py` 뒤 `grep -c` 결과가 `0` 입니다.
- `python verify.py` 가 "검증 통과"입니다.
- `git status` 에 `library.py` 가 바뀐 것으로 보이지 않습니다.

> `git restore` 는 **저장하지 않은** 변경을 마지막 저장 상태로 돌립니다. 새로 만들어진 파일은 지워지지 않으므로 `git status` 의 `Untracked files` 를 보고 직접 지웁니다.

4단계. 저장 후 변경을 git revert 로 되돌리기 (30분)

이번에는 같은 실수를 **저장한 뒤** 되돌립니다.

(1) 실수를 다시 만들고 저장하기

[컨테이너]

```
cd /workspace/library
opencode
```

■ [AI] OpenCode 입력 칸

```
library.py 에 책 삭제 기능 delete_book(books, book_id) 를 추가해 줘. 계획 없이 바로 진행해 줘.
```

`/exit` 로 나온 뒤:

[컨테이너]

```
cd /workspace/library
python verify.py
git add -A
git commit -m "실험: 범위 밖 delete_book 저장"
git log --oneline
```

확인 기준: "검증 통과"가 나오고, `git log --oneline` 맨 위에 "실험: 범위 밖 delete_book 저장"이 있습니다.

(2) 되돌리기 (기록은 남긴다)

[컨테이너]

```
cd /workspace/library
git revert --no-edit HEAD
git log --oneline
grep -c "def delete_book" library.py
```

확인 기준:

- `git log --oneline` 에 두 줄이 생겼습니다. 하나는 "실험", 다른 하나는 "Revert" 입니다. **기록은 지워지지 않았습니다.**
- `grep -c` 결과가 `0` 입니다.

> `git revert` 는 되돌린 내용을 **새 커밋**으로 만듭니다. 원래 커밋을 지우지 않습니다. 그래서 "언제 무엇을 되돌렸는지"가 기록에 남습니다.

> 주의: `git reset --hard` 는 기록을 지우므로 이 과정에서 쓰지 않습니다. 이 과정의 AI 권한 설정도 `git reset` 을 막고 있습니다.

5단계. 되돌린 뒤 검증하고 기록 확인하기 (20분)

[컨테이너]

```
cd /workspace/library
python verify.py
git status
git log --oneline
```

확인 기준:

- "검증 통과"가 나옵니다.
- `git status` 가 깨끗합니다. (`books.json` 은 `.gitignore` 에 있으므로 보이지 않는 것이 정상)
- `git log --oneline` 에 "실험" 커밋과 "Revert" 커밋이 모두 있습니다.

[Windows] `C:\harness-lab\library\library.py` 를 열어 `delete_book` 이 없는지 눈으로 확인합니다.

6단계. 되돌리기 방법 고르기 (표 채우기, 15분)

아래 상황마다 무엇을 할지 고르고, 이유를 한 문장으로 씁니다. 선택지는 `git restore` / `git revert` / 아무것도 안 함 입니다.

상황	무엇을 하나 (선택)	이유 (한 문장)
방금 AI 가 바꾼 파일을 저장하기 전에 아니라고 판단했다		
이미 저장된 커밋에 버그가 있다는 것을 어제 발견했다		
테스트가 실패하지만 원인을 아직 모른다		
3번 시도해도 검증이 통과하지 않는다		

확인 기준: 네 줄 모두 선택과 이유가 있어야 합니다. 답이 맞는지는 강사와 함께 확인합니다.

7단계. 정리하고 확인하기 (10분)

[컨테이너]

```
cd /workspace/library
rm -f CHANGE_요청서.md
git status
```

확인 기준: `git status` 에 바뀐 파일이 없어야 합니다. 실험 커밋은 **지우지 않고 그대로 둡니다.**

확인 체크리스트

- ■ `delete_book` 이 검증을 통과하는 것을 직접 봤다.
- ■ 범위 질문 두 개에 답했다. (2번 질문은 "아니오")
- ■ `git restore` 로 저장 전 변경을 되돌렸다.
- ■ `git revert` 로 저장 후 변경을 되돌렸고, 기록 두 줄을 확인했다.
- ■ 6단계 표를 채웠다.

자주 만나는 오류

증상	해결
`git revert` 가 편집기를 연다	`--no-edit` 를 빠뜨렸다. 편집기가 열리면 Esc 를 누르고 `:wq` 를 치고 Enter 를 누른다.
`git revert` 가 충돌을 알린다	되돌릴 커밋 뒤에 같은 줄을 또 고쳤다. `git revert --abort` 로 취소하고 `git log --oneline` 으로 순서를 확인한다.
`git restore` 후에도 파일이 남아 있다	새로 만들어진 파일이다. `git status` 의 `Untracked files` 에서 찾아 직접 지운다.
되돌렸는데 검증이 실패한다	되돌리기 전 다른 변경이 섞여 있었을 수 있다. `git log` 와 `git diff` 로 확인한다.

2단계 7장. 미니 프로젝트: 도서 대출 목록 완성

실행 환경: 도커. 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간 30분
준비물	1~6장 완료
결과	새 폴더에서 처음부터 도서 대출 목록을 AI 에게 맡기고, 테스트와 직접 실행으로 확인한 뒤 저장한다. 과정 기록을 쓴다

이 장은 **정답 파일을 주지 않습니다.** 요구사항과 테스트를 기준으로 확인합니다.

이 장에서 할 일 한눈에 보기

1. 새 작업 폴더를 만들고 기준 파일을 가져온다. (15분)
2. 요구사항과 테스트 이름을 대조한다. (20분)
3. AI 에게 맡기고 계획을 검토한다. (25분)
4. 검증하고 테스트 파일을 대조한다. (15분)
5. 직접 실행해서 확인한다. (20분)
6. 기록을 쓰고 저장한다. (25분)

1단계. 새 작업 폴더 만들고 기준 파일 가져오기 (15분)

[컨테이너]

```
mkdir -p /workspace/library-final/tests /workspace/library-final/.githooks
cd /workspace/library-final
cp /tutorial/templates/stage2/tests/test_books.py tests/
cp /tutorial/templates/stage2/tests/test_lending_v2.py tests/test_lending.py
cp /tutorial/templates/stage2/REQUIREMENTS.md .
cp /tutorial/templates/stage2/TASK_미니프로젝트.md .
cp /tutorial/templates/stage2/pytest.ini .
cp /tutorial/templates/verify.py .
cp /tutorial/templates/opencode.jsonc .
cp /tutorial/templates/pre-commit .githooks/pre-commit
cp /tutorial/templates/.gitattributes .
cp /workspace/library/AGENTS.md .
printf 'books.json\n.env\n' > .gitignore
git init
git config core.hooksPath .githooks
ls -la
ls tests
```

확인 기준: 아래 표의 파일이 모두 있어야 합니다.

파일	있는가? (예/아니오)
REQUIREMENTS.md	
TASK_미니프로젝트.md	
AGENTS.md	

파일	있는가? (예/아니오)
opencode.jsonc	
verify.py, pytest.ini	
tests/test_books.py, tests/test_lending.py	
.githooks/pre-commit	

확인 기준 2: `git config core.hooksPath` 가 `.githooks` 를 출력해야 합니다.

2단계. 요구사항과 테스트 이름 대조하기 (20분)

[컨테이너]

```
cd /workspace/library-final
grep -n "^### R" REQUIREMENTS.md
grep -n "^def test_" tests/test_books.py tests/test_lending.py
```

아래 표를 채웁니다. 요구사항 한 개에 테스트가 **적어도 하나** 있어야 합니다.

요구사항	요구사항의 함수 이름	이 요구사항을 확인하는 테스트 이름 (두 번째 명령의 출력에서 찾는다)
R1 책 등록	add_book	
R2 목록 보기	list_books	
R3 대출	lend_book	
R4 반납	return_book	
R5 파일 저장과 읽기	save_books, load_books	

확인 기준: 다섯 줄이 모두 채워져야 합니다. 빈칸이 있으면 그 요구사항에 테스트가 없다는 뜻입니다. `REQUIREMENTS.md` 를 고치지 말고 강사에게 알립니다.

3단계. AI 에게 맡기고 계획 검토하기 (25분)

[컨테이너]

```
cd /workspace/library-final
opencode
```

■ [AI] OpenCode 입력 칸

```
TASK_미니프로젝트.md 를 읽고 그 지시대로 작업해 줘. REQUIREMENTS.md 와 tests/ 를 기준으로 삼는다. 먼저 계획만 말하고 내 승인을 기다려.
```

계획을 아래 네 질문으로 검토합니다.

질문	답 (예/아니오)	아니오면 입력할 문장
1. 고치는 파일이 library.py 한 개뿐인가?		아래 문장 A
2. 계획에 R1~R5 의 함수가 모두 들어 있는가?		아래 문장 A
3. 계획에 책 삭제, 연체, 웹 화면, 데이터베이스가 없는가?		아래 문장 A
4. 계획이 tests/, REQUIREMENTS.md, verify.py 를 고친다고 하지 않는가?		아래 문장 A

[AI] 문장 A (아니오일 때, 번호만 바꿔 입력합니다)

```
계획 [번호]번이 TASK_미니프로젝트.md 와 맞지 않아. 그 부분을 고쳐서 계획을 다시 말해 줘. 아직 파일은 만들지 마.
```

모두 예이면 입력합니다.

■ [AI] OpenCode 입력 칸

> 진행

끝나면 `/exit` 를 칩니다.

4단계. 검증하고 테스트 파일 대조하기 (15분)

[컨테이너]

```
cd /workspace/library-final
python verify.py
diff tests/test_books.py /tutorial/templates/stage2/tests/test_books.py && echo "책 테스트 원본과 같습니다"
diff tests/test_lending.py /tutorial/templates/stage2/tests/test_lending_v2.py && echo "대출 테스트 원본과 같습니다"
git status
```

확인 기준:

확인할 것	기대하는 결과	실제 결과 (예/아니오)
검증	"검증 통과"로 끝난다 (13 passed)	
책 테스트 대조	"책 테스트 원본과 같습니다"	
대출 테스트 대조	"대출 테스트 원본과 같습니다"	
git status	새로 생긴 파일이 library.py 뿐이다	

아니오가 있으면 AI 에게 그 항목을 알리고 고치게 합니다. 테스트 파일이 바뀌었다면 원본을 다시 복사합니다.

5단계. 직접 실행해서 확인하기 (20분)

[컨테이너]

```
cd /workspace/library-final
python -c "
from library import add_book, lend_book, return_book, list_books
books = []
add_book(books, '데미안', '헤세')
add_book(books, '노인과 바다', '헤밍웨이')
lend_book(books, 1, '철수')
print('1. 대출 중 목록:', [b['title'] for b in list_books(books, 'lent')])
print('2. 대출 가능 목록:', [b['title'] for b in list_books(books, 'available')])
print('3. 반납 후:', return_book(books, 1))
try:
    return_book(books, 1)
    print('4. 대출 중이 아닌 책 반납: 오류가 없었다 (잘못됨)')
except ValueError as e:
    print('4. 대출 중이 아닌 책 반납: 오류 =', e)
"
```

확인 기준:

줄	기대하는 결과	실제 출력 (그대로 옮긴다)	맞는가? (예/아니오)
1	대출 중 목록에 "데미안"이 한 개		
2	대출 가능 목록에 "노인과 바다"가 한 개		
3	반납 후 status 는 available, borrower 는 None, loan_days 는 없다		

줄	기대하는 결과	실제 출력 (그대로 옮긴다)	맞는가? (예/아니오)
4	"오류 ="이 나온다		

아니오가 있으면 AI 에게 그 줄 번호를 알리고 고치게 합니다.

6단계. 기록 쓰고 저장하기 (25분)

[컨테이너]

```
cd /workspace/library-final
touch 기록.md
```

[Windows] VS Code 에서 `C:\harness-lab\library-final\기록.md` 를 열고 아래 틀을 붙여 넣습니다.

```
# 7장 기록

## 1. 이 프로젝트에서 바뀐 요구사항 한 가지와, 왜 기준을 사람이 바꿨는가
답:

## 2. 검증은 통과했지만 범위를 대조해서 확인한 것 한 가지
답:

## 3. 되돌리기를 했다면 어떤 방법을 썼고, 왜 그 방법을 골랐는가 (없으면 "없음")
답:

## 4. 내가 이해한 코드 세 문장
1.
2.
3.

## 5. 2단계를 마치고 1단계와 달라진 점 한 가지
답:
```

확인 기준: 모든 `답:` 뒤에 글자가 있어야 합니다.

[컨테이너]

```
cd /workspace/library-final
grep -c "^답:\s*$" 기록.md
```

`0` 이면 저장합니다. 저장은 사람이 합니다.

[컨테이너]

```
cd /workspace/library-final
python verify.py
git add -A
git status
git commit -m "7장: 도서 대출 목록 완성 (R1~R5, 검증과 범위 확인 완료)"
git log --oneline
```

확인 기준: `git status` 에 `books.json` 이 없어야 합니다. 커밋 과정에서 `[pre-commit]` 이 "검증 통과"를 출력해야 합니다.

확인 체크리스트

- ■ 1단계 파일이 모두 있고, `core.hooksPath` 가 `.githooks` 다.
- ■ 2단계 표의 다섯 줄이 모두 채워졌다.
- ■ 계획 검토 네 질문에 답했다.
- ■ `python verify.py` 가 "검증 통과"다. 테스트 파일이 원본과 같다.
- ■ 직접 실행 네 줄의 결과를 확인했다.
- ■ `기록.md` 의 `답:` 칸이 모두 채워졌다.
- ■ 커밋이 한 줄 생겼다.

막혔을 때

증상	해결
`cp` 가 "No such file" 이라고 한다	`ls /tutorial/templates/stage2` 로 파일 이름을 확인한다.
테스트가 `ImportError` 를 낸다	`library.py` 에 함수 이름이 빠졌다. 2단계 표의 이름을 AI 에게 알리고 추가하게 한다.
AI 가 `tests/` 나 `REQUIREMENTS.md` 를 고치려 한다	기준은 사람이 고친다. AI 에게 "기준 파일은 고치지 마"라고 말하고, `diff` 로 확인해 되돌린다.
직접 실행 4번 줄에 "오류가 없었다"가 나온다	대출 중이 아닌 책의 반납을 막는 검사가 없다. AI 에게 "대출 중이 아니면 ValueError 를 일으키게 해 줘"라고 말한다.

> 정답 코드는 제공하지 않습니다. 실패 줄을 하나씩 읽고, AI 와 대화하며 맞춰 갑니다.

다음 단계

이것으로 2단계가 끝납니다. 3단계는 같은 흐름을 **FastAPI 웹 API, PostgreSQL, Docker 실행 환경**으로 옮겨 가는 3일 과정입니다. 저장소 harness-crud-lab 에서 이어갑니다.

부록. 템플릿과 예제

수업에서 복사해 쓰는 파일을 그대로 실었다. 주석을 따라 읽는다.

A-1. 도서 대출 요구사항 최종본 (7장)

```
# 요구사항: 도서 대출 목록 (2단계 7장, 최종 버전)

## 한 문장 요약
책을 등록하고, 대출하고, 반납하고, 목록을 보는 명령줄 프로그램. 책 목록은 JSON 파일에 저장되어 프로그램을 껐다 켜도 남는다.

## 책 한 권의 모양
| 항목 | 설명 |
|---|---|
| `id` | 1, 2, 3 … 순서대로 붙는 번호. 지워지는 기능이 없으므로 항상 새 번호를 쓴다. |
| `title` | 책 제목. 공백만 있으면 안 된다. |
| `author` | 지은이. 비워 둘 수 있다. |
| `status` | `"available"`(대출 가능) 또는 `"lent"`(대출 중). 처음에는 `"available"`. |
| `borrower` | 빌려 간 사람 이름. 대출 중이면 이름이, 대출 가능하면 `None`. |
| `loan_days` | 대출 기간(일). 대출 중일 때만 `14` 가 들어 있다. 반납하면 이 칸을 지운다. |

## 기능 요구사항

### R1. 책 등록
- `add_book(books, title, author)` 을 부르면 새 책을 목록 끝에 넣고, 그 책을 돌려준다.
- 제목이 공백뿐이면 `ValueError` 를 일으킨다.

### R2. 목록 보기
- `list_books(books)` 은 모든 책을 돌려준다.
- `list_books(books, "available")` 또는 `list_books(books, "lent")` 은 그 상태의 책만 돌려준다.

### R3. 대출
- `lend_book(books, book_id, borrower)` 을 부르면 그 책의 `status` 를 `"lent"`, `borrower` 를 빌리는 사람 이름으로 바꾸고, `loan_days` 를 `14` 로 넣는다.
- 없는 번호면 `ValueError`.
- 이미 대출 중인 책이면 `ValueError`.
- 빌리는 사람 이름이 공백뿐이면 `ValueError`.

### R4. 반납
- `return_book(books, book_id)` 을 부르면 그 책의 `status` 를 `"available"` 로 바꾸고, `borrower` 를 `None` 으로, `loan_days` 칸을 지운다.
- 없는 번호면 `ValueError`.
- 대출 중이 아닌 책이면 `ValueError`.

### R5. 파일 저장과 읽기
- `save_books(books, path)` 은 책 목록을 JSON 파일로 저장한다. 한글이 깨지지 않는다.
- `load_books(path)` 은 JSON 파일을 읽어 목록을 돌려준다.
- 파일이 없으면 빈 목록 `[]` 을 돌려준다.

### R6. 명령줄에서 쓰기 (선택)
- `python library.py` 뒤에 `add`, `list`, `lend`, `return` 을 붙여 쓸 수 있다.
- 목록은 번호, 상태, 제목, 지은이, 빌린 사람을 한 줄씩 보여 준다.

## 만들지 않는 것
- 연체 계산, 날짜 계산 (대출 기간은 숫자 14 로만 기록한다)
- 회원 관리, 여러 사람이 동시에 쓰는 기능
- 책 삭제, 책 정보 수정
- 웹 화면, 데이터베이스
```

A-2. 요구사항 변경 요청서 (5장)

```
# 요구사항 변경 요청서 (2단계 5장)

> 요구사항이 바뀌면, 코드를 고치기 전에 이 요청서를 먼저 씁니다.
> 바뀌는 것은 사람이 정하고, 승인도 사람이 합니다. AI 는 승인된 변경만 구현합니다.

## 1. 바뀌는 요구사항
- 요구사항 번호: R3 (대출), R4 (반납)
- 무엇이 바뀌나: 대출하면 `loan_days` 칸에 `14` 가 함께 기록된다. 반납하면 그 칸을 지운다.
- 왜 바꾸나: 도서관에서 대출 기간을 알아야 한다. (이 요청의 이유를 한 문장으로 적는다)

## 2. 영향을 받는 테스트
- `tests/test_lending.py` 의 `test_lend_book_marks_lent` : 기대하는 책 모양이 달라진다.
- 그 밖의 테스트는 이 변경으로 바뀌지 않아야 한다.

## 3. 테스트 기준을 어떻게 바꾸나
- 새 기준: `templates/stage2/tests/test_lending_v2.py` 를 `tests/test_lending.py` 자리에 넣는다.
- 기준을 바꾸는 것은 **사람이 한다.** AI 에게 맡기지 않는다.

## 4. 승인
- [ ] 요청한 사람이 이 변경의 이유를 읽었다.
- [ ] 기준(테스트)을 바꾸는 일을 사람이 했다.
- [ ] 승인 날짜와 이름: _____________________

---

## 구현 지시 (승인 뒤에만 쓴다)
```

CHANGE_요청서.md 의 1번 변경을 구현해 줘. tests/ 는 이미 새 기준으로 바뀌어 있으니 고치지 마. 계획을 먼저 말하고 내가 "진행"이라고 하면 library.py 만 고쳐 줘.

A-3. 책 저장 작업 지시서 (2장)

```
# 작업 지시서: 책 등록과 파일 저장 (2단계 2장)

## 1. 목표
책을 등록하고, 목록을 보고, JSON 파일에 저장하고 읽어 올 수 있는 `library.py` 의 앞부분을 만든다.

## 2. 요구사항
`REQUIREMENTS.md` 의 **R1, R2, R5** 를 구현한다. (R3, R4 는 이번에 만들지 않는다)

## 3. 범위
- 새로 만드는 파일: `library.py` 하나
- 고쳐도 되는 파일: `library.py` 하나뿐이다.
- 고치면 안 되는 파일: `tests/` 폴더 전체, `verify.py`, `AGENTS.md`, `opencode.jsonc`

## 4. 하지 말 것
- `lend_book`, `return_book` 을 만들지 않는다. (3장에서 한다)
- 웹 화면, 데이터베이스, 외부 패키지를 쓰지 않는다. 파이썬 기본 기능만 쓴다.
- 책 삭제나 수정 기능을 덧붙이지 않는다.

## 5. 완료 조건
- [ ] `python -m pytest tests/test_books.py` 가 모두 통과한다.
- [ ] `library.py` 의 모든 함수에 한국어 주석이 있다.
- [ ] `tests/` 폴더는 바뀌지 않았다. (`Compare-Object` 로 원본과 비교했을 때 차이가 없다)

## 6. 진행 방법
`AGENTS.md` 의 작업 순서를 따른다. 먼저 계획을 말하고, 내가 "진행"이라고 하면 구현한다.
끝나면 바꾼 파일과 통과한 테스트 개수를 보고한다.
```

A-4. 대출과 반납 작업 지시서 (3장)

```
# 작업 지시서: 대출과 반납 (2단계 3장)

## 1. 목표
책을 빌려 주고 돌려받는 기능을 `library.py` 에 추가한다. 상태 규칙을 지키는 것이 핵심이다.

## 2. 요구사항
`REQUIREMENTS.md` 의 **R3, R4** 를 구현한다. (R1, R2, R5 는 이미 되어 있다)

## 3. 범위
- 고쳐도 되는 파일: `library.py` 하나뿐이다.
- 고치면 안 되는 파일: `tests/` 폴더 전체, `verify.py`, `AGENTS.md`, `opencode.jsonc`

## 4. 하지 말 것
- 연체, 날짜 계산을 만들지 않는다. (대출 기간은 이번 장에서 넣지 않는다)
- 대출 기록을 따로 파일로 남기지 않는다.
- 테스트 기대값을 바꾸거나, 테스트를 건너뛰게 만들지 않는다.

## 5. 완료 조건
- [ ] `python -m pytest tests/test_books.py tests/test_lending.py` 가 모두 통과한다.
- [ ] `tests/` 폴더는 바뀌지 않았다.
- [ ] 이미 대출 중인 책을 다시 빌리려 하면 오류가 나고, 상태는 바뀌지 않는다. (내가 직접 확인한다)

## 6. 진행 방법
`AGENTS.md` 의 작업 순서를 따른다. 먼저 계획을 말하고, 내가 "진행"이라고 하면 구현한다.
계획에 테스트 파일 수정이 들어 있으면 그 줄을 빼게 한 뒤 승인한다.
```

A-5. 미니 프로젝트 작업 지시서 (7장)

```
# 작업 지시서: 도서 대출 목록 완성 (2단계 7장)

## 1. 목표
`REQUIREMENTS.md` 의 R1~R5 를 모두 만족하는 `library.py` 를 완성한다. (R6 명령줄 기능은 선택)

## 2. 요구사항
`REQUIREMENTS.md` 의 **R1, R2, R3, R4, R5**. 대출 기간 `loan_days` 는 `CHANGE_요청서.md` 로 승인된 변경이다.

## 3. 범위
- 고쳐도 되는 파일: `library.py` 하나뿐이다.
- 고치면 안 되는 파일: `tests/` 폴더 전체, `verify.py`, `AGENTS.md`, `opencode.jsonc`, `REQUIREMENTS.md`

## 4. 하지 말 것
- 연체 계산, 날짜 계산, 책 삭제, 책 수정을 만들지 않는다.
- 웹 화면, 데이터베이스, 외부 패키지를 쓰지 않는다.
- 테스트 파일의 기대값을 바꾸지 않는다.

## 5. 완료 조건
- [ ] `python verify.py` 가 "검증 통과"로 끝난다.
- [ ] `tests/` 폴더는 템플릿 원본과 같다. (`Compare-Object` 로 확인)
- [ ] 대출 중인 책을 반납하면 `borrower` 가 `None` 이 되는지 내가 직접 확인했다.
- [ ] 모든 함수에 한국어 주석이 있다.

## 6. 진행 방법
`AGENTS.md` 의 작업 순서를 따른다. 먼저 계획을 말하고, 내가 "진행"이라고 하면 구현한다.
끝나면 `AGENTS.md` 의 보고 형식으로 보고한다.
```

A-6. 책 등록과 저장 테스트 (2장)

```
"""
test_books.py - 책 등록, 목록, 저장 약속 (2단계 2장)

이 파일이 보여 주는 개념
  1) 기준(테스트)은 사람이 정한다. AI 는 이 파일을 고칠 수 없다.
  2) 저장 테스트는 임시 폴더(tmp_path)에만 파일을 쓴다. 내 진짜 파일을 건드리지 않는다.
"""

import pytest

from library import add_book, list_books, load_books, save_books


def test_add_book_creates_available_book():
    books = []
    book = add_book(books, "데미안", "헤세")
    assert book == {"id": 1, "title": "데미안", "author": "헤세", "status": "available", "borrower": None}
    assert books == [book]


def test_add_book_numbers_go_up():
    books = []
    add_book(books, "첫째", "가")
    second = add_book(books, "둘째", "나")
    assert second["id"] == 2


def test_add_book_rejects_blank_title():
    # 공백만 있는 제목은 받지 않는다.
    with pytest.raises(ValueError):
        add_book([], "   ", "헤세")


def test_list_books_filters_by_status():
    books = []
    add_book(books, "데미안", "헤세")
    assert list_books(books) == books
    assert list_books(books, "available") == books
    assert list_books(books, "lent") == []


def test_save_and_load_roundtrip(tmp_path):
    path = tmp_path / "books.json"  # 임시 폴더 안의 파일만 쓴다
    books = []
    add_book(books, "데미안", "헤세")
    save_books(books, path)
    assert load_books(path) == books


def test_load_missing_file_returns_empty_list(tmp_path):
    # 파일이 아직 없으면 빈 목록을 돌려준다. 오류를 내지 않는다.
    assert load_books(tmp_path / "없는파일.json") == []
```

A-7. 대출과 반납 테스트 1차 (3장)

```
"""
test_lending.py - 대출과 반납 약속 (2단계 3장, 1차 버전)

이 파일이 보여 주는 개념
  1) 상태 규칙: 대출 중인 책은 다시 대출할 수 없고, 대출하지 않은 책은 반납할 수 없다.
  2) 정상 경우와 잘못된 경우를 모두 약속한다.
  3) 4교시에 "반납하면 빌린 사람 이름을 지운다"는 약속이 빠져 있다는 것을 발견하게 된다.
"""

import pytest

from library import add_book, lend_book, return_book


def make_books():
    """테스트마다 책이 한 권 있는 목록을 만든다."""
    books = []
    add_book(books, "데미안", "헤세")
    return books


def test_lend_book_marks_lent():
    books = make_books()
    book = lend_book(books, 1, "철수")
    assert book == {"id": 1, "title": "데미안", "author": "헤세", "status": "lent", "borrower": "철수"}


def test_lend_lent_book_raises():
    # 이미 대출 중인 책은 다시 빌릴 수 없다.
    books = make_books()
    lend_book(books, 1, "철수")
    with pytest.raises(ValueError):
        lend_book(books, 1, "영희")


def test_lend_missing_book_raises():
    with pytest.raises(ValueError):
        lend_book(make_books(), 99, "철수")


def test_lend_blank_borrower_raises():
    with pytest.raises(ValueError):
        lend_book(make_books(), 1, "   ")


def test_return_book_marks_available():
    books = make_books()
    lend_book(books, 1, "철수")
    book = return_book(books, 1)
    assert book["status"] == "available"


def test_return_available_book_raises():
    # 대출하지 않은 책은 반납할 수 없다.
    with pytest.raises(ValueError):
        return_book(make_books(), 1)


def test_return_missing_book_raises():
    with pytest.raises(ValueError):
        return_book(make_books(), 99)
```

A-8. 대출과 반납 테스트 2차 (5장)

```
"""
test_lending_v2.py - 대출과 반납 약속 (2단계 5장, 2차 버전: 대출 기간 추가)

5장 요구사항 변경: "대출하면 대출 기간 14일(loan_days)이 함께 기록된다.
반납하면 대출 기간 기록은 지운다."

이 파일은 사람이 승인한 뒤에만 test_lending.py 대신 쓴다.
AI 는 이 파일을 고칠 수 없다(권한 설정으로 막는다).
"""

import pytest

from library import add_book, lend_book, return_book


def make_books():
    books = []
    add_book(books, "데미안", "헤세")
    return books


def test_lend_book_marks_lent_with_loan_days():
    books = make_books()
    book = lend_book(books, 1, "철수")
    assert book == {
        "id": 1, "title": "데미안", "author": "헤세",
        "status": "lent", "borrower": "철수", "loan_days": 14,
    }


def test_return_book_removes_loan_days():
    books = make_books()
    lend_book(books, 1, "철수")
    book = return_book(books, 1)
    assert book == {"id": 1, "title": "데미안", "author": "헤세",
                    "status": "available", "borrower": None}


def test_lend_lent_book_raises():
    books = make_books()
    lend_book(books, 1, "철수")
    with pytest.raises(ValueError):
        lend_book(books, 1, "영희")


def test_lend_missing_book_raises():
    with pytest.raises(ValueError):
        lend_book(make_books(), 99, "철수")


def test_lend_blank_borrower_raises():
    with pytest.raises(ValueError):
        lend_book(make_books(), 1, "   ")


def test_return_available_book_raises():
    with pytest.raises(ValueError):
        return_book(make_books(), 1)


def test_return_missing_book_raises():
    with pytest.raises(ValueError):
```

```
        return_book(make_books(), 99)
```

A-9. 4장 연습용 초안 (검증 전)

```
"""
library.py - 도서 대출 목록 (AI 가 만든 초안, 아직 검증 전)

4장에서 이 파일을 프로젝트 폴더의 library.py 로 복사해서 검증합니다.
"""

import json
from pathlib import Path


def add_book(books, title, author):
    """책을 등록한다. 번호는 1, 2, 3 ... 순서대로 붙는다."""
    new_id = max((b["id"] for b in books), default=0) + 1
    book = {"id": new_id, "title": title.strip(), "author": author.strip(),
            "status": "available", "borrower": None}
    books.append(book)
    return book


def list_books(books, status=None):
    """책 목록을 돌려준다. status 를 주면 그 상태의 책만 돌려준다."""
    if status is None:
        return books
    return [b for b in books if b["status"] == status]


def _find(books, book_id):
    """번호로 책을 찾는다. 없으면 ValueError 를 일으킨다."""
    for b in books:
        if b["id"] == book_id:
            return b
    raise ValueError("책을 찾을 수 없습니다.")


def lend_book(books, book_id, borrower):
    """책을 대출한다. 빌리는 사람 이름을 기록한다."""
    book = _find(books, book_id)
    if not borrower.strip():
        raise ValueError("빌리는 사람 이름을 입력하세요.")
    book["status"] = "lent"
    book["borrower"] = borrower.strip()
    return book


def return_book(books, book_id):
    """책을 반납한다."""
    book = _find(books, book_id)
    if book["status"] != "lent":
        raise ValueError("대출 중이 아닌 책입니다.")
    book["status"] = "available"
    return book


def save_books(books, path):
    """책 목록을 JSON 파일로 저장한다."""
    Path(path).write_text(json.dumps(books, ensure_ascii=False, indent=2), encoding="utf-8")


def load_books(path):
    """JSON 파일을 읽는다. 파일이 없으면 빈 목록을 돌려준다."""
    p = Path(path)
    if not p.exists():
```

```
        return []
    return json.loads(p.read_text(encoding="utf-8"))
```